Registro de trazabilidad de la conversación

Proyecto: API de usuarios temporales con NestJS

Este registro resume, en orden cronológico, las consultas realizadas durante el desarrollo del proyecto, el objetivo de cada una, la información obtenida y la relación entre consultas consecutivas. Solo se incluyen interacciones que aparecen en la conversación.

N.°	Consulta o prompt	Objetivo	Información y palabras clave obtenidas	Cómo condujo a la siguiente consulta
1	Solicitud de un proyecto completo en NestJS desde cero, sin Prisma ni base de datos, para una API de usuarios temporales (id, nombre, email, edad). Se plantearon nueve etapas y la regla de avanzar una por una, entregando primero solo la Etapa 1.	Definir el alcance del proyecto, las herramientas permitidas y la metodología de trabajo por etapas.	Etapa 1 entregada: nest new usuarios-api, cd usuarios-api, code ., npm run start:dev y verificación en http://localhost:3000 (“Hello World!”). Palabras clave: NestJS, proyecto, Visual Studio Code, Postman, memoria, etapas.	Con el proyecto base explicado, la persona solicitó recibir el resto de los pasos sin esperar la confirmación etapa por etapa.
2	“Ya pero dame todo los pasos”.	Obtener el procedimiento completo (Etapas 2 a 9) en una sola respuesta.	Estructura de carpetas (src, main.ts, app.module.ts, users/). Comandos nest g module / controller / service users --no-spec. Código de UsersModule, UsersController y UsersService con arreglo en memoria. Métodos HEAD y OPTIONS (decoradores @Head, @Options, @Header('Allow')). Script scripts/crear-usuario.js con fetch y comando npm run usuario, como solución al problema de memoria separada entre procesos. Lista de pruebas (Etapa 8) y flujo Terminal → NestJS → Controller → Service → Usuarios → Postman (Etapa 9).	Al intentar la Etapa 4, la persona no comprendió cómo operar Postman y pidió una explicación detallada de esa etapa.
3	“Como hago la etapa 4 no entiendo explícame porfa”.	Comprender el uso de Postman para crear y consultar usuarios.	Guía paso a paso: método POST, URL http://localhost:3000/users, pestaña Body, raw, JSON, header Content-Type: application/json y botón Send. Respuesta esperada 201 Created con el usuario y su id. Pruebas de GET, GET por id y DELETE, además de una tabla de errores comunes.	Con POST resuelto, la persona avanzó a la prueba de HEAD y preguntó cómo abrir una pestaña adicional en Postman.

N.°	Consulta o prompt	Objetivo	Información y palabras clave obtenidas	Cómo condujo a la siguiente consulta
4	“Como creo otra pestaña para el Head?”.	Aprender a abrir una nueva pestaña en Postman y configurar el método HEAD.	Botón +, atajo Ctrl + T o New → HTTP. Selección del método HEAD, URL http://localhost:3000/users y revisión de la pestaña Headers de la respuesta. Resultado esperado: 200 OK, cuerpo vacío y encabezado X-Total-Count.	La persona ejecutó las pruebas de POST, HEAD y OPTIONS y envió capturas de pantalla para validar los resultados.
5	Envío de tres capturas de Postman (sin texto adicional): OPTIONS /users con 404, POST /users con 201 y HEAD /users con 200.	Verificar si las pruebas de HEAD y OPTIONS funcionaban correctamente.	POST correcto (201 Created, id 1). HEAD con 200 OK, pero sin el encabezado X-Total-Count (Content-Length: 2). OPTIONS con 404 Not Found (el servidor respondía “Cannot OPTIONS /users”). Diagnóstico: los métodos @Head y @Options no estaban activos en users.controller.ts. Se entregó el archivo users.controller.ts completo para reemplazar el existente.	Tras reemplazar el controller, la persona repitió las pruebas y envió una nueva captura de OPTIONS.
6	Envío de una captura de Postman (sin texto adicional): OPTIONS /users/1.	Confirmar que la corrección del controller había funcionado.	Respuesta 204 No Content con Allow: GET, HEAD, DELETE, OPTIONS. Confirmación de que @Options(':id') estaba activo. Se indicó repetir HEAD /users y OPTIONS /users para completar la verificación.	La persona volvió a probar HEAD y OPTIONS sobre /users y envió dos capturas más.
7	Envío de dos capturas de Postman (sin texto adicional): HEAD /users y OPTIONS /users.	Validar las pruebas pendientes de HEAD y OPTIONS en la colección.	HEAD /users: 200 OK con X-Total-Count: 1 y cuerpo vacío. OPTIONS /users: 204 No Content con Allow: GET, HEAD, POST, OPTIONS. Etapas 4, 5 y 6 confirmadas; resumen de pasos de la Etapa 7 (script de terminal y npm run usuario).	Con las Etapas 4 a 6 comprobadas, la persona pidió aclarar el funcionamiento de la Etapa 8.
8	“De la etapa 8 como funciona?”.	Entender el procedimiento de las pruebas integrales.	Reinicio del servidor para vaciar la memoria. Siete pruebas en orden: POST, GET por id, creación desde terminal (id 2), GET de todos los usuarios, HEAD, OPTIONS y DELETE. La prueba 4 demuestra que Postman y la terminal comparten los mismos usuarios. Tabla de errores comunes (servidor apagado, script no definido, reinicio por guardado).	Con el proceso técnico documentado, la persona solicitó un registro de trazabilidad de la conversación.

N.°	Consulta o prompt	Objetivo	Información y palabras clave obtenidas	Cómo condujo a la siguiente consulta
9	Solicitud de un registro resumido de las consultas realizadas, con objetivo, información obtenida y vínculo entre consultas, en una tabla en formato Word, sin inventar interacciones.	Documentar el proceso seguido durante la conversación como evidencia de trazabilidad.	Este documento: tabla con las nueve consultas efectuadas, ordenadas cronológicamente. Palabras clave: trazabilidad, consultas, objetivo, secuencia.	No aplica (última consulta registrada).